



UNIVERSIDAD CATÓLICA BOLIVIANA "SAN PABLO"

DEPARTAMENTO DE CIENCIAS DE LA TECNOLOGÍA E INNOVACIÓN

INGENIERÍA MECATRÓNICA

Controlador Multimedia Inalámbrico Bluetooth basado en ESP32

INGENIERÍA MECATRÓNICA



Materia: Sistemas Embebidos II

Estudiante(s): Mario Alberto Nina Gallo

Docente: Ing. Elmer Alan Cornejo Quito

Semestre: 1/2026

Bolivia, 8 de junio de 2026

Índice

1. Introducción	1
1.1. Contexto y motivación	1
1.2. Objetivos	1
2. Marco teórico	2
2.1. Bluetooth Low Energy y perfil HID	2
2.2. ESP32 y ESP-IDF	2
2.3. NimBLE	3
2.4. FreeRTOS	3
3. Diseño e implementación del hardware	3
3.1. Arquitectura del sistema	3
3.2. Cadena de alimentación	3
3.3. Interfaz de usuario	4
3.4. Diseño del PCB	4
3.5. Asignación de pines	4
4. Diseño e implementación del firmware	5
4.1. Arquitectura de componentes	5
4.2. Driver de entrada	5
4.3. Comunicación Bluetooth HID	6
4.4. Máquina de estados	6
4.5. Gestión de energía	6
4.5.1. Modo de bajo consumo	7
5. Resultados y validación	7
5.1. Validación funcional	7
5.2. Alcance Bluetooth	7
5.3. Autonomía de batería	8
5.4. Latencia	8
5.5. Análisis de resultados	8
6. Conclusiones	8
Referencias Bibliográficas	10

Índice de figuras

1.	Diagrama en bloques del sistema.	4
----	--	---

1. Introducción

El presente informe documenta el diseño, implementación y validación de un controlador multimedia inalámbrico portátil basado en el microcontrolador ESP32 de Espressif Systems. El sistema permite gestionar la reproducción de contenido de audio en una computadora portátil sin requerir interacción física directa con la misma, mediante comunicación inalámbrica Bluetooth.

1.1. Contexto y motivación

El control remoto de reproducción multimedia es una necesidad frecuente en contextos donde el usuario se encuentra en posición de descanso o realizando actividades que dificultan el acceso físico al dispositivo de reproducción. Las soluciones comerciales existentes presentan limitaciones significativas: los controles infrarrojos requieren línea de vista directa y receptores dedicados; las soluciones propietarias están limitadas a ecosistemas cerrados; y las alternativas de radiofrecuencia genérica requieren dongles USB que ocupan puertos y limitan la portabilidad del sistema.

Este proyecto aborda dicha problemática mediante el desarrollo de un dispositivo que implementa el protocolo Bluetooth Human Interface Device (HID), garantizando compatibilidad nativa con el sistema operativo sin necesidad de controladores o software adicional.

1.2. Objetivos

Objetivo general: Diseñar e implementar un controlador multimedia inalámbrico portátil basado en microcontrolador de la familia ESP32 que permita gestión remota de reproducción de audio en computadora mediante comunicación Bluetooth, con operación autónoma mediante batería recargable.

Objetivos específicos:

1. Implementar circuito electrónico en PCB basado en microcontrolador ESP32, integrando sistema de alimentación con batería recargable y circuito de carga con protecciones.
2. Desarrollar firmware en C mediante ESP-IDF que gestione comunicación Bluetooth y procesamiento de comandos multimedia: Play/Pause, Next, Previous, volumen, y Mute/Unmute.
3. Establecer comunicación inalámbrica Bluetooth con sistema operativo Linux para transmisión de comandos multimedia.
4. Implementar interfaz de usuario que permita activación diferenciable de funciones, con retroalimentación visual de estado operativo.

5. Validar funcionalmente el sistema mediante pruebas de alcance, autonomía y confiabilidad de transmisión.

2. Marco teórico

2.1. Bluetooth Low Energy y perfil HID

Bluetooth Low Energy (BLE) es una tecnología de comunicación inalámbrica diseñada para operar en la banda ISM de 2.4 GHz con consumo de energía significativamente reducido respecto a Bluetooth clásico (Bluetooth Special Interest Group, 2023). Opera mediante 40 canales de 2 MHz con salto de frecuencia adaptativo, ofreciendo un rango típico de hasta 50 metros en condiciones favorables.

El perfil HID over GATT (HoGP) es una adaptación del protocolo USB HID para operar sobre BLE mediante el Generic Attribute Profile (GATT) (Bluetooth Special Interest Group, 2011). Este perfil define cómo un dispositivo BLE puede exponer servicios HID a un host, permitiendo que el sistema operativo lo reconozca como un dispositivo de entrada estándar sin necesidad de controladores adicionales. El perfil requiere la implementación de tres servicios GATT: el servicio HID (UUID 0x1812), el servicio de información del dispositivo (UUID 0x180A), y el servicio de batería (UUID 0x180F).

Los comandos multimedia se transmiten mediante la página de uso Consumer Control (Usage Page 0x0C) del estándar USB HID Usage Tables (USB Implementers Forum, 2023). Cada comando se identifica por un código de uso de 16 bits: Play/Pause (0x00CD), Next Track (0x00B5), Previous Track (0x00B6), Volume Increment (0x00E9), Volume Decrement (0x00EA), y Mute (0x00E2).

2.2. ESP32 y ESP-IDF

El ESP32 es un sistema en chip (SoC) de Espressif Systems que integra un procesador Xtensa LX6 de doble núcleo a 240 MHz, memoria RAM de 520 KB, y controladores de radio para WiFi 802.11 b/g/n y Bluetooth 4.2/5.0 (Espressif Systems, 2024). Su arquitectura dual-core permite ejecutar el stack Bluetooth en un núcleo mientras el firmware de aplicación opera en el otro, evitando interferencias entre ambos.

ESP-IDF (Espressif IoT Development Framework) es el framework oficial de desarrollo para los SoCs de Espressif (Espressif Systems, 2026). Proporciona una capa de abstracción sobre el hardware, drivers de periféricos, y componentes de sistema incluyendo FreeRTOS, NVS (Non-Volatile Storage), y gestión de energía mediante `esp_pm`.

2.3. NimBLE

NimBLE es un stack Bluetooth Low Energy de código abierto desarrollado por el proyecto Apache Mynewt (Apache Software Foundation, 2025). Es el stack BLE integrado en ESP-IDF y se caracteriza por su reducida huella de memoria respecto al stack clásico de Espressif, su soporte completo del rol de periférico BLE, y su compatibilidad con perfiles GATT estándar. NimBLE implementa todas las capas del stack BLE: capa física, enlace, L2CAP, ATT, GATT, GAP y el gestor de seguridad (SM).

2.4. FreeRTOS

FreeRTOS es un sistema operativo de tiempo real diseñado para microcontroladores, integrado en ESP-IDF (Real Time Engineers Ltd., 2016). Proporciona primitivas de sincronización y comunicación entre tareas, incluyendo colas (*queues*), semáforos y mutex. En este proyecto se utiliza para gestionar la comunicación entre el driver de entrada y la tarea de procesamiento de comandos, así como para proteger el acceso concurrente al estado del sistema desde múltiples núcleos del ESP32.

3. Diseño e implementación del hardware

3.1. Arquitectura del sistema

El sistema se compone de dos elementos principales: el controlador portátil basado en ESP32 y la computadora objetivo. La comunicación entre ambos se establece mediante protocolo Bluetooth BLE HID. El controlador integra cuatro subsistemas: interfaz de usuario, microcontrolador ESP32, sistema de alimentación e indicadores de estado, cuya interacción se ilustra en la figura 1.

3.2. Cadena de alimentación

El sistema de alimentación implementa una cadena de conversión de energía que parte de una celda Li-ion 18650 de 3.7V y 2600 mAh. El módulo TP4056 gestiona la carga de la batería mediante conexión USB-C, incorporando protección contra sobrecarga, sobredescarga y cortocircuito. Dado que el voltaje nominal de la batería (3.7V) puede variar entre 3.0V y 4.2V durante el ciclo de carga y descarga, se incorporó un boost converter MT3608 para elevar y estabilizar el voltaje a 5V. Finalmente, el regulador AMS1117-3.3V reduce el voltaje a los 3.3V requeridos por el ESP32 y los demás componentes del sistema.

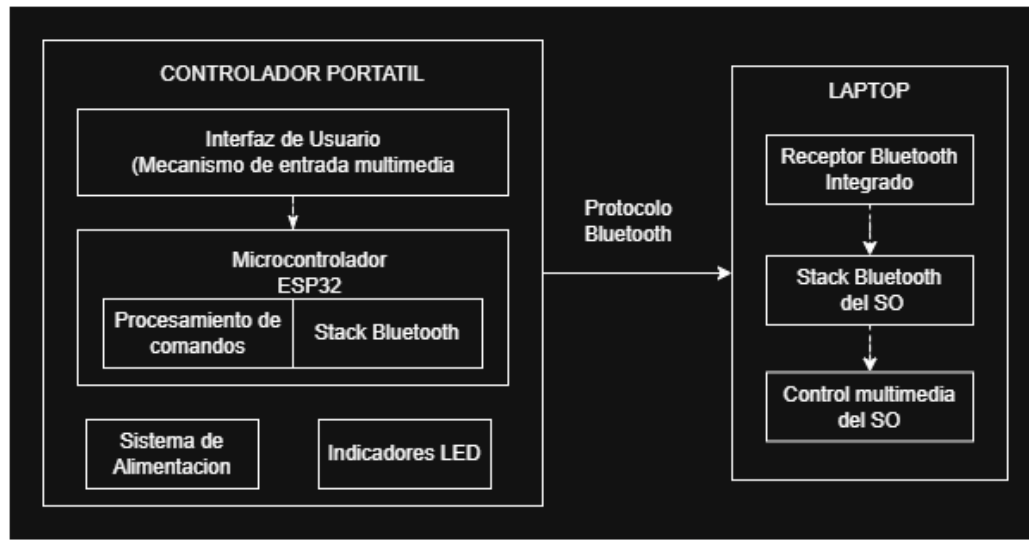


Figura 1: Diagrama en bloques del sistema.

3.3. Interfaz de usuario

La interfaz de usuario integra un encoder rotativo Alps EC11 y tres pulsadores táctiles. El encoder permite control de volumen mediante giro (CW para incremento, CCW para decremento) y Play/Pause mediante pulsación de su eje. Los tres botones activan las funciones de siguiente pista, pista anterior y Mute/Unmute respectivamente. Dos LEDs proveen retroalimentación visual: el LED azul indica el estado de la conexión Bluetooth y el LED rojo indica el nivel de batería.

3.4. Diseño del PCB

El PCB se diseñó en KiCad 8.0 como una placa de una capa de 70×100 mm fabricada mediante transferencia térmica sobre baquelita. Los anchos de pista se determinaron mediante la norma IPC-2221, resultando en 0.8 mm para señales y 1.0 mm para líneas de potencia, con un clearance mínimo de 0.5 mm adecuado para el proceso de fabricación casero. Se implementó un plano de cobre completo en la cara de cobre para la red GND, reduciendo la resistencia de retorno y simplificando el ruteo. La disposición de componentes sigue un criterio ergonómico vertical: la interfaz de usuario en la zona superior, el ESP32 en la zona media, y la cadena de potencia en la zona inferior.

3.5. Asignación de pines

La selección de pines del ESP32 siguió criterios de compatibilidad con BLE activo, disponibilidad de periféricos hardware dedicados, y ausencia de funciones de strapping que pudieran afectar el arranque. Los pines GPIO32 y GPIO33 se asignaron a los LEDs por pertenecer a ADC1, siendo compatibles con BLE activo. Los pines GPIO18 y GPIO19 se

asignaron al encoder por su soporte nativo del periférico PCNT. El pin GPIO34 se asignó a la lectura ADC de batería por ser entrada exclusiva perteneciente a ADC1, compatible con BLE activo. Los pines ADC2 se evitaron completamente dado su incompatibilidad con el stack BLE.

4. Diseño e implementación del firmware

4.1. Arquitectura de componentes

El firmware se desarrolló en lenguaje C utilizando ESP-IDF v6.0.1, estructurado en componentes independientes bajo el sistema de construcción CMake de ESP-IDF. Cada componente encapsula una responsabilidad específica del sistema, siguiendo el principio de separación de responsabilidades.

- **common:** Configuración global del sistema mediante `config.h` y mapa de pines GPIO mediante `pines.h`.
- **driver_entrada:** Driver de botones con debouncing por timestamp e interrupción GPIO, y driver de encoder rotativo mediante el periférico PCNT con decodificación de cuadratura en hardware.
- **comunicacion_bt:** Stack NimBLE con advertising BLE, callbacks de conexión y desconexión, y perfil HID Consumer Control sobre GATT.
- **control_leds:** Control de patrones de iluminación LED mediante temporizadores `esp_timer` sin bloqueo de tareas.
- **gestion_energia:** Monitoreo periódico de voltaje de batería mediante ADC1 con promediado de muestras y calibración line fitting.

4.2. Driver de entrada

El driver de botones configura los GPIOs con pull-up interno e interrupción por flanco descendente (`GPIO_INTR_NEGEDGE`). El debouncing se implementa por software mediante comparación de timestamps con `esp_timer_get_time()`, descartando eventos que ocurren dentro de un intervalo de 150 ms respecto al evento anterior. Este enfoque es más robusto que el debouncing por retardo (`vTaskDelay`) ya que no bloquea el contexto de la ISR.

El driver de encoder utiliza el periférico PCNT (Pulse Counter) del ESP32 para decodificación de cuadratura en hardware, configurando dos canales para capturar flancos en ambas señales CLK y DT. Los watch points se configuran en ± 4 pulsos, lo que corresponde a exactamente un detente físico del EC11, generando un evento de subida o bajada de volumen por cada clic perceptible del encoder. Un filtro de glitches de 10 μ s suprime el ruido mecánico del encoder antes de que alcance el contador.

4.3. Comunicación Bluetooth HID

El stack NimBLE se inicializa con BLE clásico desactivado para reducir el consumo de memoria flash. El servicio HID se registra en el servidor GATT mediante un descriptor HID que define el formato de reporte Consumer Control: un campo de 16 bits con rango 0x0000 a 0x03FF correspondiente a los códigos de uso de la página Consumer Control del estándar USB HID Usage Tables.

Para cada comando, se envían dos reportes consecutivos mediante notificaciones GATT: el reporte activo con el código de uso correspondiente, seguido obligatoriamente de un reporte de key-up (0x0000). Este segundo reporte es necesario para indicar al host que la tecla fue liberada; sin él, el sistema operativo interpreta una pulsación continua y ejecuta la acción repetidamente.

Al producirse una desconexión, el stack reinicia el advertising de manera inmediata. Dado que el ESP32 opera como periférico BLE, no puede iniciar conexiones — es el host quien debe conectarse al periférico. Este comportamiento es correcto para dispositivos HID: Ubuntu reconoce el dispositivo como de confianza y reconecta automáticamente al detectar el advertising.

4.4. Máquina de estados

El sistema implementa una máquina de estados finitos con dos estados operativos:

- **SYS_ADVERTISING:** Estado inicial y de espera. El LED azul parpadea rápidamente indicando que el dispositivo está anunciando su presencia mediante BLE advertising.
- **SYS_CONNECTED:** Conexión BLE activa. El LED azul se apaga. Los comandos de entrada se procesan y transmiten al host.

El acceso al estado del sistema está protegido mediante un mutex FreeRTOS, dado que el ESP32 es un sistema dual-core donde la tarea NimBLE opera en el núcleo 1 y la tarea de procesamiento de comandos opera en el núcleo 0, existiendo posibilidad de acceso concurrente a la variable de estado.

4.5. Gestión de energía

El monitoreo de batería se implementa mediante lectura del ADC1 canal 6 (GPIO34), que mide el voltaje en el punto medio de un divisor resistivo 100k Ω /100k Ω conectado a la batería. El voltaje leído se multiplica por 2 para obtener el voltaje real de la batería. Se promedian 16 muestras consecutivas para reducir el ruido inherente del ADC, y se aplica calibración line fitting de ESP-IDF para obtener valores en milivoltios con mayor precisión.

Se definieron tres niveles de batería: OK (≥ 3400 mV), bajo (≤ 3400 mV) y crítico (≤ 3200 mV), con retroalimentación visual correspondiente mediante el LED rojo.

4.5.1. Modo de bajo consumo

Durante el desarrollo se evaluó la implementación de light sleep para reducir el consumo energético durante períodos de inactividad. Sin embargo, se determinó que el stack NimBLE en la configuración utilizada pierde la conexión BLE activa al entrar en light sleep, dado que el CPU se suspende y no puede mantener los intervalos de conexión requeridos por el protocolo BLE.

Como solución alternativa se implementó modem sleep mediante `esp_pm_configure()`, habilitando el escalado dinámico de frecuencia del CPU entre 40 MHz y 160 MHz. Este modo permite al sistema reducir la frecuencia de operación automáticamente cuando las tareas están en espera, sin interrumpir la conexión BLE. Si bien el ahorro energético es menor que el del light sleep, esta solución mantiene la conexión activa en todo momento, lo que es esencial para la experiencia de uso del dispositivo.

5. Resultados y validación

5.1. Validación funcional

Se realizó una prueba de integración completa con el sistema en hardware físico, validando los seis comandos multimedia con Spotify y YouTube bajo Ubuntu 24.04 LTS. Los resultados se presentan en la tabla 1.

Tabla 1: Resultados de validación de comandos multimedia.

Comando	Control físico	Resultado
Play/Pause	Encoder — presionar	Funciona correctamente
Siguiente pista	Botón 1	Funciona correctamente
Pista anterior	Botón 2	Funciona correctamente
Subir volumen	Encoder — girar derecha	Funciona correctamente
Bajar volumen	Encoder — girar izquierda	Funciona correctamente
Mute/Unmute	Botón 3	Funciona correctamente

5.2. Alcance Bluetooth

Se verificó el funcionamiento del dispositivo en condiciones de uso típico dentro de una habitación, con y sin obstáculos entre el controlador y la computadora. El enlace BLE se mantuvo estable en el rango de operación de una habitación grande, cumpliendo el objetivo de permitir control desde ubicaciones de descanso sin necesidad de línea de vista directa.

5.3. Autonomía de batería

Con la celda Li-ion 18650 de 2600 mAh completamente cargada, el dispositivo operó durante un mínimo de 4 horas en uso normal, definido como comandos ejecutados aproximadamente cada 5 minutos con el sistema en estado conectado. La sesión de prueba se interrumpió antes del agotamiento de la batería, por lo que la autonomía real podría ser superior.

5.4. Latencia

La respuesta del sistema fue evaluada subjetivamente durante las sesiones de uso. El retardo entre la activación de un control y la ejecución de la acción correspondiente en el host fue imperceptible en condiciones normales de operación, consistente con la naturaleza de baja latencia del protocolo BLE HID para comandos discretos.

5.5. Análisis de resultados

Los resultados obtenidos validan el cumplimiento de los objetivos planteados. El sistema implementa exitosamente el perfil BLE HID Consumer Control, logrando compatibilidad nativa con Ubuntu 24.04 LTS sin necesidad de software adicional. La autonomía de 4 horas con una celda de 2600 mAh representa una mejora significativa respecto a la batería LiPo de 500 mAh inicialmente contemplada en el diseño.

La decisión de reemplazar light sleep por modem sleep implicó un compromiso entre ahorro energético y continuidad de la conexión BLE. Para un dispositivo HID, mantener la conexión activa es prioritario sobre el máximo ahorro energético, ya que una desconexión requiere intervención del usuario para reconectar. El modem sleep con escalado dinámico de frecuencia CPU provee un balance adecuado para el caso de uso del proyecto.

6. Conclusiones

Se diseñó e implementó exitosamente un controlador multimedia inalámbrico portátil basado en el microcontrolador ESP32, cumpliendo los objetivos planteados al inicio del proyecto.

El sistema implementa el perfil Bluetooth HID Consumer Control mediante el stack NimBLE sobre ESP-IDF v6.0.1, logrando compatibilidad nativa con Linux sin necesidad de controladores adicionales. Los seis comandos multimedia planteados — Play/Pause, Next, Previous, Volume Up, Volume Down y Mute/Unmute — operan correctamente de extremo a extremo desde el hardware físico hasta el sistema operativo.

El diseño del firmware siguió principios de ingeniería de software embebido: separación de responsabilidades en componentes independientes, centralización de configuración, protección de recursos compartidos mediante primitivas FreeRTOS, y cero números mágicos en el código. Estas decisiones facilitan el mantenimiento y la extensión del sistema.

El proceso de desarrollo evidenció la importancia de la validación física incremental: decisiones como la selección de pines GPIO, la configuración del periférico PCNT para el encoder, y la elección del modo de bajo consumo surgieron de pruebas en hardware real y no pudieron determinarse únicamente por análisis teórico.

Como trabajo futuro se identifican las siguientes líneas de mejora: implementación de light sleep coordinado con el stack BLE para mayor ahorro energético, diseño de PCB multicapa para reducir el tamaño del dispositivo, y extensión de la compatibilidad a sistemas operativos Windows y macOS.

Referencias Bibliográficas

- Apache Software Foundation. (2025). *Apache NimBLE — BLE User Guide*. Apache Software Foundation. Consultado el 7 de junio de 2026, desde <https://mynewt.apache.org/latest/network/>
- Bluetooth Special Interest Group. (2011). *HID over GATT Profile Specification 1.0*. Bluetooth SIG. Consultado el 7 de junio de 2026, desde <https://www.bluetooth.com/specifications/specs/hid-over-gatt-profile-1-0/>
- Bluetooth Special Interest Group. (2023). *Bluetooth Core Specification 5.4*. Bluetooth SIG. Consultado el 7 de junio de 2026, desde <https://www.bluetooth.com/specifications/specs/core-specification-5-4/>
- Espressif Systems. (2024). *ESP32 Series Datasheet*. Espressif Systems. Consultado el 7 de junio de 2026, desde https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- Espressif Systems. (2026). *ESP-IDF Programming Guide v6.0.1*. Espressif Systems. Consultado el 7 de junio de 2026, desde <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/index.html>
- Real Time Engineers Ltd. (2016). *FreeRTOS Reference Manual*. Real Time Engineers Ltd. Consultado el 7 de junio de 2026, desde https://www.freertos.org/Documentation/RTOS_book.html
- USB Implementers Forum. (2023). *HID Usage Tables for USB 1.5*. USB Implementers Forum. Consultado el 7 de junio de 2026, desde https://usb.org/sites/default/files/hut1_5.pdf